

Problema 1

```
for (i = n/2; i <= n; i++)  
  for (j = 1; j+n/2 <= n; j++)  
    for (k = 1; k <= n; k = k*2) counter++;
```

Paso 1 ciclo i

$$i = n/2, \dots, n$$

$$\text{vueltas} = n - n/2 + 1 = n/2 + 1 \quad \rightarrow O(n)$$

Paso 2 ciclo j

$$j + n/2 \leq n \Rightarrow j \leq n/2$$

$$j = 1, \dots, n/2$$

$$\text{vueltas} = n/2 \quad \rightarrow O(n)$$

Paso 3 ciclo k

$$k = 1, 2, 4, 8, \dots \Rightarrow k = 2^m$$

$$2^m > n \Rightarrow m > \log_2(n)$$

$$\text{vueltas} = \log_2(n) + 1 \quad \rightarrow O(\log n)$$

Paso 4 estan anidados $\Rightarrow$ se multiplican

$$T(n) = (n/2 + 1) (n/2) (\log_2 n + 1)$$

$$= n^2 \log_2(n) / 4 + n^2/4 + n \log_2(n) / 2 + n/2$$

$$\text{dominante: } n^2 \log_2(n) / 4$$

$$T(n) = O(n^2 \log n)$$

comprobacion P1 (counter)

$$n = 100 \quad 51 \times 50 \times 7 = 17,850$$

$$n = 1,000 \quad 501 \times 500 \times 10 = 2,505,000$$

$$n = 10,000 \quad 5001 \times 5000 \times 14 = 350,070,000$$

Problema 2

if ($n \leq 1$) return n ;

for ($i = 1; i \leq n; i++$)

for ($j = 1; j \leq n; j++$) { printf; break; }

Paso 1 guarda

if ($n \leq 1$) return

-> $O(1)$

Paso 2 ciclo i

$i = 1, \dots, n$

vuelatas = n

-> $O(n)$

Paso 3 ciclo j

$j = 1$ -> printf -> break

vuelatas = 1 (no n)

-> $O(1)$

Paso 4 $T(n) = O(1) + n \times 1 = n + 1$

$T(n) = O(n)$

ojo: son 2 for anidados pero NO es $O(n^2)$

el break corta el ciclo interno

imprime "Sequence" exactamente n veces

Problema 3

```
for (i = 1; i <= n/3; i++)  
    for (j = 1; j <= n; j += 4) printf;
```

Paso 1 ciclo i

$$i = 1, \dots, n/3 \quad \text{vueltas} = n/3 \quad \rightarrow O(n)$$

Paso 2 ciclo j

$$j = 1, 5, 9, \dots \Rightarrow j = 1 + 4m$$

$$1 + 4m > n \Rightarrow m > (n-1)/4$$

$$\text{vueltas} = (n-1)/4 + 1 \sim n/4 \quad \rightarrow O(n)$$

Paso 3 están anidados $\Rightarrow$ se multiplican

$$T(n) = (n/3)(n/4) = n^2 / 12$$

$T(n) = O(n^2)$

comprobación

$n = 1,000$	333×250	$= 83,250$	$n^2/12 = 83,333$
$n = 10,000$	3333×2500	$= 8,332,500$	$n^2/12 = 8,333,333$

Resumen

Problema 1 $\rightarrow O(n^2 \log n)$

Problema 2 $\rightarrow O(n)$

Problema 3 $\rightarrow O(n^2)$

Problema 4

Busqueda Lineal

recorre el arreglo de izquierda a derecha
hasta encontrar x

Mejor caso

x esta en la primera posición
1 comparación, no depende de n

$$T(n) = \Theta(1)$$

Peor caso

x esta al final, o no esta en el arreglo
 n comparaciones

$$T(n) = \Theta(n)$$

Caso promedio

x esta en la posición i con probabilidad $1/n$

$$E[T] = \sum_{i=1}^n i \cdot (1/n)$$

$$= (1/n) \cdot n(n+1)/2$$

$$= (n+1)/2 \quad \rightarrow \text{la mitad del arreglo}$$

$$T(n) = \Theta(n)$$

Problema 4

Busqueda Binaria

el arreglo ya esta ordenado y en cada paso se descarta la mitad

Mejor caso

x cae justo en el medio al primer intento

$$T(n) = \Theta(1)$$

Peor caso

$$T(n) = T(n/2) + 1 \quad T(1) = 1$$

desenrollando k veces:

$$T(n) = T(n / 2^k) + k$$

$$\text{para } n / 2^k = 1 \Rightarrow k = \log_2(n)$$

$$T(n) = \log_2(n) + 1$$

$$T(n) = \Theta(\log n)$$

Caso promedio

el nivel i del arbol tiene 2^i elementos

y cada uno cuesta $i+1$ comparaciones

$$E[T] = (1/n) \sum 2^i (i+1) \sim \log_2(n) - 1$$

$$T(n) = \Theta(\log n)$$

Problema 4

Quick Sort

particiona alrededor de un pivote y ordena
las dos partes por separado

Mejor caso

el pivote parte el arreglo por la mitad

$$T(n) = 2 T(n/2) + \Theta(n)$$

Teorema Maestro: $a=2, b=2, f(n)=n$

$$n^{(\log_2 2)} = n = f(n) \quad \rightarrow \text{caso 2}$$

$$T(n) = \Theta(n \log n)$$

Peor caso

el pivote siempre es el menor (o el mayor):
arreglo ya ordenado

$$T(n) = T(n-1) + \Theta(n)$$

$$= n + (n-1) + \dots + 1 = n(n+1)/2$$

$$T(n) = \Theta(n^2)$$

Caso promedio

$$T(n) = (2/n) \sum_{i=0}^{n-1} T(i) + \Theta(n)$$

$$\text{resolviendo: } T(n) \sim 1.39 n \log_2(n)$$

$$T(n) = \Theta(n \log n)$$

Problema 5

$$a) \quad f = \Theta(g) \quad y \quad g = \Theta(h) \quad \Rightarrow \quad h = \Theta(f)$$

$$f = \Theta(g) \Rightarrow c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$g = \Theta(h) \Rightarrow c_3 h(n) \leq g(n) \leq c_4 h(n)$$

sustituyendo, para $n \geq \max(n_1, n_2)$:

$$c_1 c_3 h(n) \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \leq c_2 c_4 h(n)$$

$$\Rightarrow f = \Theta(h)$$

despejando $h(n)$:

$$(1/c_2 c_4) f(n) \leq h(n) \leq (1/c_1 c_3) f(n)$$

$$\Rightarrow h = \Theta(f)$$

VERDADERO

Θ es una relación de equivalencia;

reflexiva, simétrica y transitiva

Problema 5

$$b) \quad f = O(g) \quad y \quad g = O(h) \quad \Rightarrow \quad h = \Omega(f)$$

$$f = O(g) \Rightarrow f(n) \leq c_1 g(n)$$

$$g = O(h) \Rightarrow g(n) \leq c_2 h(n)$$

encadenando:

$$f(n) \leq c_1 g(n) \leq c_1 c_2 h(n)$$

$$\Rightarrow f = O(h)$$

despejando $h(n)$:

$$h(n) \geq (1/c_1 c_2) f(n)$$

$$\Rightarrow h = \Omega(f)$$

VERDADERO

$f = O(h)$ y $h = \Omega(f)$ son lo mismo

visto desde cada lado

Problema 5

c) $f(n) = \Theta(n^2)$ para el Programa:

```
atupla = tuple(range(0, n))
```

```
for i in range(0, n):
```

```
    for j in range(i+1, n):
```

```
        S.add(atupla[i:j])
```

Las Vueltas si son $\Theta(n^2)$, pero el cuerpo

NO es $\Theta(1)$:

atupla[i:j] copia una tupla de largo j-i

y hashearla para el set cuesta otro j-i

$$\text{costo} = \sum_{i=0..n-1} \sum_{j=i+1..n-1} (j - i)$$

con $k = j - i$:

$$\sum_{k=1..n-1-i} k = (n-1-i)(n-i) / 2$$

$$\text{total} = \sum_{m=0..n-1} m(m+1)/2 \sim n^3/6$$

$$f(n) = \Theta(n^3), \text{ no } \Theta(n^2)$$

FALSO